Gorch on Microcontrollers

int8 Inference Kernel and simavr Test Harness

Branch `claude/gorch-microcontroller-inference-Uw9vI`

2026-04-17

Summary

This report documents a prototype that demonstrates gorch-style neural network inference on an 8-bit microcontroller (ATmega2560, the chip behind Arduino Mega), and a fully-automated test harness that runs on any Linux host with the AVR toolchain installed. The harness uses **simavr** to simulate the MCU and asserts that the firmware's int8 inference output matches a Go-side reference kernel byte-for-byte.

All code lives under `embedded/` on branch `claude/gorch-microcontroller-inference-Uw9vI`.

What was built

1. `gm1_avr.h` — the device kernel

A header-only C file (~100 lines) providing three ops:

- **`gm1_linear_i8`**: fully-connected layer, int8 weights × int8 activations, int32 accumulator, fused requantize back to int8.
- **`gm1_relu_i8`**: in-place ReLU.
- **`gm1_linear_i32`**: classifier tail, produces int32 logits so argmax runs on full precision.
- **`gm1_argmax_i32`**: trivially, returns the index of the largest logit.

Design choices:

- **Weights in flash** via `PROGMEM` / `pgm_read_byte`. AVR Harvard architecture requires this; without it even tiny MLPs blow out 8 KB of SRAM.
- **Per-layer fused requant multiplier** `M` (Q0.31) and shift `S` in the style of TFLite's `MultiplyByQuantizedMultiplier`: a single 64-bit multiply plus a shift plus saturation, no floats needed on a chip with no FPU.
- **Host/device parity via macros**: the same header compiles on AVR (PROGMEM reads) and on the host (plain dereferences), so a Go-side reference can match the C exactly.

2. `reference.go` / `model.go` — Go-side mirror + model type

- `reference.go` duplicates every arithmetic step of the C kernel in Go, using the same integer widths and rounding rule (half-up at shift position). This is what the harness compares firmware output against.

- `model.go` defines `TinyModel` (a list of `Layer` records) and emits both `tiny_model.h` (weights, biases, `M`/`S` constants as PROGMEM arrays) and `tiny_input.h` (the test vector).

3. `firmware/firmware.c` — ATmega2560 fixture

A small `main()` that reads `GM1_INPUT` from flash, runs `LinearI8 → ReLU → LinearI32 → argmax`, and prints `CLASS <n> LOGITS <l0> <l1> ...` over UART0.

4. `avr_runner/runner.c` — the simulation driver

The stock `simavr` CLI does not, in the Debian 1.6 packaging, install any hook that pipes firmware I/O to stdout. This ~60-line host program uses **libsimavr** directly:

1. `elf_read_firmware` parses the ELF (including the `.mmcu` section that `AVR_MCU(...)` places into the binary).
2. `avr_make_mcu_by_name` + `avr_init` + `avr_load_firmware` bring up the simulated core.
3. `avr_io_getirq(avr, AVR_IOCTL_UART_GETIRQ('0'), UART_IRQ_OUTPUT)` and `avr_irq_register_notify` wire up a callback on UART0's TX that forwards each transmitted byte to `stdout`.
4. The driver runs the sim until the CPU halts, a cycle budget is reached, or a newline has been printed and a short grace period elapses — the latter being the normal exit condition, since the firmware spins after its single output line.

5. `avr_harness_test.go` — end-to-end Go test

```
TestAVRInferenceMatchesReference
  for each of 4 deterministic inputs:
    regenerate firmware/tiny_model.h
    regenerate firmware/tiny_input.h
    make -C firmware clean all       # → firmware.elf
    ./avr_runner/avr_runner firmware.elf
    parse "CLASS n LOGITS ..."
    assert equal to model.Infer(input)
```

The test `t.Skip`s cleanly when `avr-gcc` or `make` is missing, so CI on machines without the AVR toolchain still passes.

What the discovery process looked like

The design of the harness was not the interesting part — the interesting part was getting simavr to emit bytes. Three dead ends, in order, were:

1. **`AVR_MCU_SIMAVR_CONSOLE(&GPIOR0)` on the stock `simavr` CLI.** The macro places a record in the ELF's `.mmcu` section pointing at GPIOR0. `libsimavr`'s `elf_read_firmware` parses it into `fw.console_register_addr`, but the CLI `simavr` binary never calls `avr_set_console_register`. Verified by `strings` + `nm`: the symbol exists in `libsimavr`, but not in `simavr` itself.
2. **Calling `avr_set_console_register` from a custom runner, pointing at a `.noinit` variable in plain SRAM.** `avr_register_io_write` aborts (`SIGABRT`) when asked to watch a non-I/O address, because the IO write table it indexes is bounded by the I/O register range. Confirmed via gdb backtrace.

3. **Calling `avr_set_console_register` at GPIOR0.** This does not abort, but GPIOR0 is in the *low* I/O range, so GCC emits the `out 0x1e, r24` instruction rather than `sts`. The simavr watch fires only on memory stores; `out` writes bypass it. Silently, no output.

The fix was to drop the console-register approach entirely and use a **real UART0** in the firmware plus a **`UART_IRQ_OUTPUT` notify hook** in the host runner. This is the idiomatic simavr integration and works reliably across MCUs.

Result

```
$ go test -run TestAVRInferenceMatchesReference -v
=== RUN   TestAVRInferenceMatchesReference
=== RUN   TestAVRInferenceMatchesReference/input_0
=== RUN   TestAVRInferenceMatchesReference/input_1
=== RUN   TestAVRInferenceMatchesReference/input_2
=== RUN   TestAVRInferenceMatchesReference/input_3
--- PASS: TestAVRInferenceMatchesReference (1.16s)
PASS
ok      github.com/vinq1911/gorch/embedded  1.165s
```

Each subtest:

- emits a fresh pair of C headers (random 4→6→3 int8 weights + a specific input vector),
- rebuilds `firmware.elf` against them,
- runs the firmware under `libsimavr`,
- reads `CLASS n LOGITS l0 l1 l2` off the simulated UART,
- asserts those bytes match the Go reference `Infer(input)`.

Total wall time: ~1.2 seconds, of which ~50 ms is the AVR simulation itself; the rest is `avr-gcc` invocations.

Footprint

For the 4→6→3 toy model used by the harness:

Resource	Used	ATmega2560 budget
Flash (code + weights)	~1 KB	256 KB
SRAM (ping-pong + logits)	~30 B	8 KB
Inference	« 1 ms @ 16 MHz	—

For a realistic MNIST 784→64→10 MLP (the next-step demo):

Resource	Used	ATmega2560 budget
Flash (weights alone)	~51 KB	256 KB
SRAM (two 784-byte buffers + logits)	~1.6 KB	8 KB
Inference	~20 ms @ 16 MHz	—

What this does not yet include

1. **A gorch → gm1 exporter.** The harness today builds models in pure Go without reference to gorch's `*nn.Linear`. Because gorch is `//go:build darwin`-gated, the exporter will live in a separate file behind the same build tag, take trained `nn.Module`s, run a calibration batch to record per-layer activation ranges, quantize weights to int8, compute the fused `M/S`, and emit the same header format this harness already consumes. Once that exists, the same test shape will validate real trained models.
2. **A released-binary (`.hex`) target.** The Makefile produces `firmware.elf`; adding `avr-objcopy -O ihex` for flashing real hardware is a one-line change.
3. **CMSIS-NN / ESP-DSP fast paths.** The kernel is portable C. On Cortex-M you would swap `gm1_linear_i8` for `arm_fully_connected_s8` once an activation calibration pass is wired in.

File listing

```
embedded/
  README.md                    documentation
  .gitignore                   excludes generated headers + binaries
  gm1_avr.h                    device kernel (host-portable)
  reference.go                 Go mirror of gm1_avr.h
  model.go                     TinyModel + C-header emitters
  avr_harness_test.go          end-to-end test
  avr_runner/
    Makefile
    runner.c                   libsimavr driver + UART hook
  firmware/
    Makefile
    firmware.c                 ATmega2560 test fixture
```

Commit: `eef0f1d` on branch `claude/gorch-microcontroller-inference-Uw9vI`.